

Python Project GranTED

sgiani95

main.py

```
print(open('main.py').read())
```

```
# main.py: Entry Point for GranTED - Orchestrates Full Workflow
```

```
import argparse
```

```
import sys
```

```
from pathlib import Path
```

```
import json
```

```
# Local importsyes
```

```
from data_io import DataLoader
```

```
from preprocess import preprocess_pipeline
```

```
from gran_functions import compute_gran_functions
```

```
from analyzer import analyze_gran
```

```
from visualizer import visualize_all
```

```
from reporter import generate_report
```

```
# pycallgraph imports
```

```
from pycallgraph2 import PyCallGraph
```

```
from pycallgraph2 import Config
```

```
from pycallgraph2 import GlobbingFilter
```

```
from pycallgraph2.output import GraphvizOutput
```

```

def main():
    parser = argparse.ArgumentParser(description="GranTED Titration Analysis")
    parser.add_argument('--data_file', default='data.dat', help='Path to data file')
    parser.add_argument('--config_file', help='Path to JSON config file (optional)')
    parser.add_argument('--V', type=float, help='Initial volume offset (mL, overrides config)')
    parser.add_argument('--C_B', type=float, help='Titrant concentration (M, overrides config)')
    parser.add_argument('--titration_type', default=None, choices=['strong_acid', 'weak_acid', 'strong_base', 'weak_base'], help=
    parser.add_argument('--output_dir', default='./output', help='Output directory for plots/reports')
    args = parser.parse_args()

    # Step 1: Load data
    loader = DataLoader()
    df = loader.load_single_file(args.data_file)
    if df is None:
        sys.exit(1)

    # Step 2: Preprocess (load config)
    config_overrides = {}
    if args.V is not None:
        config_overrides['V'] = args.V
    if args.C_B is not None:
        config_overrides['C_B'] = args.C_B
    if args.titration_type is not None:
        config_overrides['titration_type'] = args.titration_type

    df_processed, params = preprocess_pipeline(df, config_overrides, args.config_file)
    print("Preprocessed data ready. Final params:", params)

    # Step 3: Compute Gran functions
    gran_results = compute_gran_functions(df_processed, params)
    print("Gran functions computed.")

    # Step 4: Analyze (interval ID and k optimization)
    analysis_results = analyze_gran(df_processed, params)
    print("Analysis complete.")

```

```

# Step 5: Visualize
visualize_all(df_processed, params, analysis_results, args.output_dir)
print("Visualizations generated.")

# Step 6: Report
generate_report(df_processed, params, analysis_results, args.output_dir)
print(f"Full workflow complete. Results in {args.output_dir}")

if __name__ == "__main__":
    config = Config()
    config.trace_filter = GlobbingFilter(include=['analyzer.*', 'compare_gran_schwartz.*', 'data_io.*', 'dynamic_callgraph.*', 'g
    graphviz = GraphvizOutput(output_file='full_callgraph.svg', output_type='svg')
    with PyCallGraph(output=graphviz, config=config):
        main()

```

data_io.py

```
print(open('data_io.py').read())
```

```

# data_io.py: Module 1 for GranTED - Data Input and Validation

#####
# Core Functionality: #
#####
#
# Loading: Supports single files (dat/txt/csv/xlsx) or batch directories, parsing columns ('volume', 'potential' in mV).
#
# Validation: Ensures monotonic increasing volumes and potential in -420 to 420 mV (pH 0-14 equivalent); flags issues but doesn't
#
# Output: Returns clean DataFrame(s) for preprocess.py; handles errors gracefully (e.g., missing files, wrong format).

import pandas as pd
from pathlib import Path

```

```

class DataLoader:
    def __init__(self, default_format='dat'):
        self.default_format = default_format # e.g., 'dat', 'csv', 'xlsx'
        self.supported_formats = ['dat', 'txt', 'csv', 'xlsx']

    def load_single_file(self, file_path, columns=['volume', 'potential']):
        """
        Load a single titration file.
        Args:
            file_path (str): Path to the file.
            columns (list): Expected columns (default: ['volume', 'potential']).
        Returns:
            pd.DataFrame: Loaded data, or None if invalid.
        """
        path = Path(file_path)
        if not path.exists():
            print(f"Error: File '{file_path}' not found.")
            return None

        try:
            if path.suffix in ['.dat', '.txt', '.csv']:
                df = pd.read_csv(file_path, sep='\s+', names=columns, header=None)
            elif path.suffix == '.xlsx':
                df = pd.read_excel(file_path, names=columns, header=None)
            else:
                print(f"Unsupported format: {path.suffix}. Use {self.supported_formats}.")
                return None

            # Basic shape check
            if df.shape[1] < 2:
                print("Error: File must have at least 2 columns (volume, potential).")
                return None

            print(f"Loaded {len(df)} points from '{file_path}'.")
            print(df.head())
            return df

```

```

except Exception as e:
    print(f"Error loading '{file_path}': {e}")
    return None

def load_batch_files(self, directory_path):
    """
    Load multiple files from a directory.
    Args:
        directory_path (str): Directory containing .dat/.csv files.
    Returns:
        dict: {filename: DataFrame} for valid files.
    """
    path = Path(directory_path)
    if not path.exists():
        print(f"Error: Directory '{directory_path}' not found.")
        return {}

    files = {}
    for file in path.glob('*.dat'):
        df = self.load_single_file(file)
        if df is not None:
            files[str(file.name)] = df

    print(f"Loaded {len(files)} files from '{directory_path}'.")
    return files

def validate_data(self, df):
    """
    Validate DataFrame: monotonic volume, reasonable potential.
    Args:
        df (pd.DataFrame): Data with 'volume' and 'potential' columns.
    Returns:
        bool: True if valid.
    """
    if df is None or len(df) < 3:
        print("Error: DataFrame too small or None.")

```

```

        return False

    if not df['volume'].is_monotonic_increasing:
        print("Warning: Volumes not strictly increasing.")
        return False

    # Check reasonable potential values (-420 to 420 mV, approx pH 0-14, see Nernst equation  $\text{pH} = 7 - (E / 59.16)$ )
    if (df['potential'] < -420).any() or (df['potential'] > 420).any():
        print("Warning: Potential values outside typical range (-420 to 420 mV).")

    print(f"Validated: {len(df)} points, volume range {df['volume'].min():.2f}-{df['volume'].max():.2f} mL")
    return True

# Example usage (for testing)
if __name__ == "__main__":
    loader = DataLoader()
    df = loader.load_single_file('data.dat')
    if df is not None:
        loader.validate_data(df)

    # Batch example
    # batch = loader.load_batch_files('./titrations/')
    # print("Batch files:", list(batch.keys()))

```

preprocess.py

```
print(open('preprocess.py').read())
```

preprocess.py: Module 2 for GranTED - Data Preprocessing and Parameter Setup

```
#####
```

```
# Core Functionality: #
```

```
#####
```

```
#
```

```
# Parameter Configuration: Loads and validates user parameters (e.g., V=25 mL, C_B=0.1 M, titration_type='weak_acid')
```

```

# via hybrid input (CLI, JSON file, or interactive prompts), ensuring defaults for monoprotic acid-
base titrations.
#
# Data Preparation: Converts raw DataFrame (from data_io.py) to NumPy arrays (volume, potential, pH via mV-
to-pH:  $\text{pH} = 7 - (\text{potential}/59.16)$ ),
# merging params for downstream use.
#
# Output: Returns the raw DataFrame and a params dict for gran_functions.py/analyzer.py; no transformation.

import numpy as np
import json
import pandas as pd

def get_config_from_cli_or_file_or_prompt(config_file=None, args=None):
    """
    Hybrid parameter loading: CLI (argparse), file (JSON), or interactive prompts.
    Args:
        config_file (str): Path to JSON config file (optional).
        args (dict): CLI args from argparse (optional).
    Returns:
        dict: Config with defaults (V=25, C_B=0.1, titration_type='weak_acid', etc.).
    """
    config = {
        'V': 25.0, # Initial volume offset (mL)
        'C_B': 0.1, # Titrant concentration (M)
        'r2_threshold': 0.95, # For linearity checks
        'titration_type': 'weak_acid', # 'strong_acid', 'weak_acid', 'strong_base', 'weak_base'
    }

    # Load from file if provided
    if config_file:
        with open(config_file, 'r') as f:
            file_config = json.load(f)
            config.update(file_config)
        print(f"Loaded config from {config_file}")

```

```

# Override with CLI args if provided
if args:
    config.update(vars(args))
    print("Applied CLI overrides")

# Interactive prompts for missing keys
for key in ['V', 'C_B', 'titration_type']:
    if key not in config or config[key] is None:
        default = config.get(key, 'default')
        value = input(f"Enter {key} (default {default}): ") or str(default)
        config[key] = value if key == 'titration_type' else float(value)

# Validate titration params
if config['titration_type'] not in ['strong_acid', 'weak_acid', 'strong_base', 'weak_base']:
    print("Warning: titration_type must be 'strong_acid', 'weak_acid', 'strong_base', or 'weak_base'.")
    config['titration_type'] = 'weak_acid' # Fallback

print("Final config:", config)
return config

def preprocess_pipeline(df, config_overrides=None, config_file=None):
    """
    Main preprocessing routine: Prepare arrays, set params.
    Args:
        df (pd.DataFrame): Raw data with 'volume' and 'potential' columns.
        config_overrides (dict): CLI overrides (optional).
        config_file (str): Path to JSON config (optional).
    Returns:
        tuple: (df, params_dict).
    """
    config = get_config_from_cli_or_file_or_prompt(config_file, config_overrides)

    # Prepare arrays for analyzer
    params = {
        'volume': df['volume'].values,
        'potential': df['potential'].values,
    }

```

```

    }
    params.update(config) # Merge with user params

    print("Preprocessing complete. Ready for analysis.")
    return df, params

# Example usage (for testing)
if __name__ == "__main__":
    df = pd.read_csv('data.dat', sep='\s+', names=['volume', 'potential'])
    df_processed, params = preprocess_pipeline(df, config_file='config.json')
    print("Processed params:", params)

```

gran_functions.py

```
print(open('gran_functions.py').read())
```

```
# gran_functions.py: Module 3 for GranTED - Gran Function Computation
```

```
#####
```

```
# Core Functionality: #
```

```
#####
```

```
#
```

```
# Gran Computation: Calculates Gran functions for strong/weak acid/base titrations based on titration_type.
```

```
# Supports 4 cases: strong_acid_g1, weak_acid_g1, strong_base_g1, weak_base_g1, with single tunable k.
```

```
#
```

```
# pH Conversion: Converts 'potential' (mV) to pH = 7 - (potential / 59.16) for calculations.
```

```
#
```

```
# Output: Returns dict {'g1': array, 'gran_func': callable(v, ph, k)} for analyzer.py; screened dicts if k_range provided.
```

```
# Note: Default titration_type='weak_acid'; set via params (CLI/JSON/GUI).
```

```
import numpy as np
```

```
import pandas as pd
```

```
def compute_gran_functions(df, params, k_range=None):
```

```
    """
```

```

Compute Gran function for the specified titration type (acid/base, strong/weak).
Args:
    df (pd.DataFrame): Data with 'volume' and 'potential' columns.
    params (dict): From preprocess.py (e.g., {'V': 25.0, 'titration_type': 'weak_acid'}).
    k_range (list): Optional k values for screening (placeholder).
Returns:
    dict: {'g1': array, 'gran_func': callable(v, ph, k)} for optimization; screened if k_range.
"""
# Ensure numeric types
volume = pd.to_numeric(df['volume'], errors='coerce').to_numpy()
potential = pd.to_numeric(df['potential'], errors='coerce').to_numpy()
pH = 7 - (potential / 59.16) # Convert mV to pH

V = float(params.get('V', 25.0))
titration_type = params.get('titration_type', 'weak_acid') # Default weak_acid
k_default = float(params.get('k', 0.0)) # Default k for computation

# Select Gran function based on titration_type
if titration_type == 'strong_acid':
    g1 = (volume + V) * np.power(10, k_default - pH)
    gran_func = lambda v, ph, k: (v + V) * np.power(10, k - ph)
    print(f"Using strong_acid_g1 with V={V}, k={k_default}")

#####

elif titration_type == 'weak_acidddd':
    # Case 1 (Schwartz 1987): Moderately weak acid
    # Corrected Gran plot accounting for non-negligible [H+]

    H = np.power(10, -pH) # [H+]
    Vtot = V + volume # total volume
    u_c = volume + ((k_default + 2500)* H) # corrected volume

    g1 = H * u_c

    gran_func = lambda v, ph, k: (

```

```

        np.power(10, -ph) *
        (v + ((V + v) * np.power(10, -ph) / 0.1))
    )

    print("Using Case 1 modified Gran function (moderately weak acid)")

#####

    elif titration_type == 'weak_acid':
        g1 = volume * np.power(10, k_default - pH)
        gran_func = lambda v, ph, k: v * np.power(10, k - ph)
        print(f"Using weak_acid_g1 with k={k_default}")
    elif titration_type == 'strong_base':
        g1 = (volume + V) * np.power(10, k_default + pH)
        gran_func = lambda v, ph, k: (v + V) * np.power(10, k + ph)
        print(f"Using strong_base_g1 with V={V}, k={k_default}")
    elif titration_type == 'weak_base':
        g1 = volume * np.power(10, k_default + pH)
        gran_func = lambda v, ph, k: v * np.power(10, k + ph)
        print(f"Using weak_base_g1 with k={k_default}")
    else:
        raise ValueError(f"Unknown titration_type '{titration_type}'; use 'strong_acid', 'weak_acid', 'strong_base', or 'weak_base'")

    results = {
        'g1': g1, # Computed array with default k
        'gran_func': gran_func, # Callable for analyzer optimization
    }

    # Placeholder for k-screening (TODO: Enable for multiple k trials)
    if k_range is not None:
        results['g1_screened'] = {}
        for k in k_range:
            results['g1_screened'][f'k={k}'] = gran_func(volume, pH, k)

    print(f"Computed Gran function for {titration_type} with default k={k_default}")
    return results

```

analyzer.py

```
print(open('analyzer.py').read())
```

```
# analyzer.py: Module 4 for GranTED - Interval Identification and Parameter Optimization
```

```
#####
```

```
# Core Functionality: #
```

```
#####
```

```
#
```

```
# Interval Identification: Automatically detects "the Zone" linear region in the Gran function (g1) using derivative analysis + r  
# targeting the steep negative-slope part for reliable fitting. Includes optional SavGol smoothing based on raw derivative noise
```

```
#
```

```
# Parameter Optimization: Tunes k5 to maximize  $R^2$  in the identified interval, with optional extension to include more points while
```

```
#
```

```
# Analysis Orchestration: Integrates Gran computation, interval finding, and optimization, outputting results (interval indices,  
# for visualizer.py and reporter.py.
```

```
#
```

```
# Output: Dict with {'interval': (start, end), 'unoptimized': {...}, 'optimized': {...}, 'g1': array}, ready for plotting/reporting
```

```
import numpy as np
```

```
import pandas as pd
```

```
from scipy.stats import linregress
```

```
from scipy.signal import savgol_filter
```

```
from gran_functions import compute_gran_functions
```

```
from scipy.optimize import minimize_scalar
```

```
def _compute_r2(g1_smooth, volume, start, end):
```

```
    """Helper: Compute  $R^2$  and fit for a segment."""
```

```
    if end - start < 2:
```

```
        return 0.0, 0.0, 0.0
```

```
    slope, intercept, r_value, _, _ = linregress(volume[start:end], g1_smooth[start:end])
```

```
    return r_value**2, slope, intercept
```

```
def identify_linear_interval(g1, volume, min_points=5, window_size=5, noise_threshold=0.001,  
                             var_threshold_rel=0.1, min_r2=0.95):
```

```

"""
Identify "the Zone" using derivative-based segmentation and ranking.
Handles Cases 1-3 via plateau detection in negative dg1.
"""
# Step 1: Compute dg1 (with optional smoothing)
dg1_raw = np.gradient(g1, volume)
std_dg1 = np.std(dg1_raw)
if std_dg1 > noise_threshold:
    print(f"High noise (std_dg1={std_dg1:.2e} > {noise_threshold}). Smoothing.")
    g1_smooth = savgol_filter(g1, window_length=window_size, polyorder=2)
    dg1 = np.gradient(g1_smooth, volume)
else:
    print(f"Low noise (std_dg1={std_dg1:.2e} <= {noise_threshold}). No smoothing.")
    g1_smooth = g1
    dg1 = dg1_raw

# Focus on negative dg1 region
neg_mask = dg1 < 0
if not np.any(neg_mask):
    print("Warning: No negative derivative. Using full range.")
    return 0, len(g1) - 1, 0.0

dg1_neg = dg1[neg_mask]
vol_neg_idx = np.where(neg_mask)[0]

# Step 2: Segment into candidate plateaus via sliding-window variance
candidates = []
win_sizes = range(max(3, min_points//2), min(15, len(dg1_neg)//2) + 1)
for win_len in win_sizes:
    for i in range(len(dg1_neg) - win_len + 1):
        seg_dg1 = dg1_neg[i:i+win_len]
        mean_dg = np.mean(seg_dg1)
        var_dg = np.var(seg_dg1)
        if mean_dg < 0 and var_dg < var_threshold_rel * abs(mean_dg):
            orig_start = vol_neg_idx[i]
            orig_end = vol_neg_idx[i + win_len - 1] + 1

```

```

        candidates.append({
            'orig_start': orig_start,
            'orig_end': orig_end,
            'length': win_len,
            'mean_dg': mean_dg,
            'var_dg': var_dg
        })

if not candidates:
    print("No low-var negative segments found. Falling back to full negative region.")
    full_start = vol_neg_idx[0]
    full_end = vol_neg_idx[-1] + 1
    _, _, r2_full = _compute_r2(g1_smooth, volume, full_start, full_end)
    return full_start, full_end, r2_full

# Step 3: Rank candidates by  $R^2$ , length, |mean_dg|
scored = []
for cand in candidates:
    r2, slope, intercept = _compute_r2(g1_smooth, volume, cand['orig_start'], cand['orig_end'])
    if r2 >= min_r2:
        score = r2 * 100 + (cand['length'] / len(g1)) * 10 + abs(cand['mean_dg']) * 0.1
        scored.append((cand, score, r2))

if not scored:
    # Fallback: Score all by composite even if  $R^2 < \min\_r2$ 
    fallback_scored = []
    for cand in candidates:
        r2, _, _ = _compute_r2(g1_smooth, volume, cand['orig_start'], cand['orig_end'])
        score = r2 * 100 + (cand['length'] / len(g1)) * 10 + abs(cand['mean_dg']) * 0.1
        fallback_scored.append((cand, score, r2))
    scored = fallback_scored

best_cand, best_score, best_r2 = max(scored, key=lambda x: x[1])
start_idx, end_idx = best_cand['orig_start'], best_cand['orig_end']

# Case labeling (heuristic)

```

```

case = "Case 3" # Default gradual
if start_idx > 0:
    pre_mean = np.mean(dg1[:start_idx])
    pre_std = np.std(dg1[:start_idx])
    if pre_mean > 0:
        case = "Case 1"
    elif pre_std < 0.01 * abs(pre_mean):
        case = "Case 2"

print(f"Identified 'the Zone': indices {start_idx}-{end_idx}, R²={best_r2:.3f} (Case: {case}, Score: {best_score:.2f})")
return start_idx, end_idx, best_r2

def _compute_fit(df, start, end, gran_func, k5=0.0):
    """
    Compute fit, R², and V_eq for a zone with given k5, using gran_func callable.
    """
    volume = df['volume'].iloc[start:end].values
    potential = df['potential'].iloc[start:end].values
    pH = 7 - (potential / 59.16)
    y = gran_func(volume, pH, k5) # Use callable
    slope, intercept, r_value, _, _ = linregress(volume, y)
    r2 = r_value**2
    veq = -intercept / slope if slope != 0 else np.nan
    return {'r2': r2, 'fit': (slope, intercept), 'veq': veq}

def _optimize_single_zone(df, start, end, gran_func, k_bounds):
    """
    Optimize k5 for a fixed zone using gran_func callable.
    """
    volume = df['volume'].iloc[start:end].values
    potential = df['potential'].iloc[start:end].values
    pH = 7 - (potential / 59.16)

    def negative_r2(k5):
        y = gran_func(volume, pH, k5) # Use callable
        slope, intercept, r_value, _, _ = linregress(volume, y)

```

```

        return -r_value**2

    result = minimize_scalar(negative_r2, bounds=k_bounds, method='bounded')
    best_k5 = result.x
    max_r2 = -result.fun
    y_opt = gran_func(volume, pH, best_k5)
    slope, intercept, _, _, _ = linregress(volume, y_opt)
    return {'best_k5': best_k5, 'best_r2': max_r2, 'fit': (slope, intercept)}

def analyze_gran(df, params, use_segmented=True, use_extension=True):
    """
    Main analysis: Compute Gran, identify raw interval, fit raw, then optimize/extend for opt Zone.
    Returns results with separate raw/opt Zones for comparison.
    """
    # Compute Gran functions (weakacid_g1 focus, with k5=0 default)
    gran_results = compute_gran_functions(df, params)
    g1 = gran_results['g1'] # General g1 array (weak_acid_g1 by default)
    gran_func = gran_results['gran_func'] # Extract callable for optimization
    params['volume'] = df['volume'].values # Ensure volume in params for visualizer

    # Identify initial interval (on raw g1)
    if use_segmented:
        start_idx, end_idx, _ = identify_linear_interval(g1, params['volume'])
    else:
        start_idx, end_idx, _ = _identify_linear_original(g1, params['volume'])

    initial_interval = (start_idx, end_idx)

    # Raw Zone: Fit on initial interval (no opt/extension)
    raw_zone = _compute_fit(df, start_idx, end_idx, gran_func, k5=0.0)
    raw_zone['start'] = start_idx
    raw_zone['end'] = end_idx
    raw_zone['num_points'] = end_idx - start_idx

    print(f"Raw Zone (initial): indices {start_idx}-{end_idx}, R²={raw_zone['r2']:.4f}, V_eq={raw_zone['veq']:.3f} mL over {raw_z")

```

```

# Opt k5 on raw Zone (no extension yet)
k_bounds = (-10, 10) # Default bounds
opt_k5 = _optimize_single_zone(df, start_idx, end_idx, gran_func, k_bounds)['best_k5']

# Recompute g1_opt and re-detect Zone on it
pH_full = 7 - (df['potential'].values / 59.16)
g1_opt = gran_func(params['volume'], pH_full, opt_k5)
if use_segmented:
    opt_start, opt_end, opt_interval_r2 = identify_linear_interval(g1_opt, params['volume'])
else:
    opt_start, opt_end, opt_interval_r2 = _identify_linear_original(g1_opt, params['volume'])

# Opt fit on re-detected Zone
opt_zone = _compute_fit(df, opt_start, opt_end, gran_func, k5=opt_k5)
opt_zone['k5'] = opt_k5
opt_zone['start'] = opt_start
opt_zone['end'] = opt_end
opt_zone['num_points'] = opt_end - opt_start

# Fallback if opt Zone smaller than raw (prevent shrinkage)
if opt_zone['num_points'] < raw_zone['num_points']:
    opt_start, opt_end = raw_zone['start'], raw_zone['end']
    opt_zone = _compute_fit(df, opt_start, opt_end, gran_func, k5=opt_k5)
    opt_zone['k5'] = opt_k5 # Ensure k5 in fallback
    opt_zone['start'] = opt_start
    opt_zone['end'] = opt_end
    opt_zone['num_points'] = opt_end - opt_start
    print(f"Opt fallback to raw Zone (to avoid shrinkage): R²={opt_zone['r2']:.4f}, points={opt_zone['num_points']}")

print(f"Opt Zone (final): k5={opt_k5:.3f}, R²={opt_zone['r2']:.4f}, V_eq={opt_zone['veq']:.3f} mL over {opt_zone['num_points']}")

results = {
    'raw_zone': raw_zone, # Initial raw
    'opt_zone': opt_zone, # Re-detected or fallback opt
    'g1': g1, # Raw g1 for plotting
    'g1_opt': g1_opt, # Opt g1 for plotting

```

```

        'interval_r2': raw_zone['r2'], # Legacy
    }

    print("Gran analysis complete with separate raw/opt Zones.")
    return results

def _identify_linear_original(g1, volume, min_points=5, window_size=5):
    """
    Original non-segmented interval identification (fallback).
    """
    dg1_raw = np.gradient(g1, volume)
    std_dg1 = np.std(dg1_raw)
    if std_dg1 > 0.001:
        g1_smooth = savgol_filter(g1, window_length=window_size, polyorder=2)
        dg1 = np.gradient(g1_smooth, volume)
    else:
        g1_smooth = g1
        dg1 = dg1_raw
    negative_start = np.where(dg1 < 0)[0]
    if len(negative_start) == 0:
        return 0, len(g1) - 1, 0.0
    start_idx = negative_start[0]
    min_deriv_idx = start_idx + np.argmin(dg1[start_idx:])
    best_r2 = 0.0
    best_start, best_end = start_idx, min_deriv_idx
    for test_start in range(start_idx, min_deriv_idx - min_points + 1):
        for test_end in range(test_start + min_points, min(min_deriv_idx + 10, len(g1))):
            if test_end - test_start < min_points:
                continue
            slope, intercept, r_value, _, _ = linregress(volume[test_start:test_end], g1_smooth[test_start:test_end])
            r2 = r_value**2
            if r2 > best_r2:
                best_r2 = r2
                best_start, best_end = test_start, test_end
    print(f"Original method: indices {best_start}-{best_end}, R²={best_r2:.3f}")
    return best_start, best_end, best_r2

```

```
# Example usage (for testing)
if __name__ == "__main__":
    df = pd.read_csv('data.dat', names=['volume', 'potential'], sep='\s+')
    params = {'V': 25.0}
    results = analyze_gran(df, params)
    print("Results:", results)
```

visualizer.py

```
print(open('visualizer.py').read())
```

```
# visualizer.py: Module 5 for GranTED - Plotting and Visualization
```

```
#####
```

```
# Core Functionality: #
```

```
#####
```

```
#
```

```
# Plot Generation: Creates Matplotlib plots for titration curves (volume vs. pH from mV),
```

```
# Gran functions (raw and opt g1 in separate panels with Zones/fits, no hardcoding), and
```

```
# screened k-values (multiple lines for comparison). Derivative subplot autoscale focuses on negative part.
```

```
#
```

```
# Customization: Supports output directories, filenames, and annotations (e.g., slope/intercept labels); uses seaborn style for plots
```

```
#
```

```
# Orchestration: Single entry visualize_all calls all plots, saving PNGs (300 DPI) to a specified dir.
```

```
#
```

```
# Output: High-res PNG files for reports/papers; extensible for Gnuplot 3D or batch summaries.
```

```
import matplotlib.pyplot as plt
```

```
import numpy as np
```

```
from pathlib import Path
```

```
from scipy.signal import savgol_filter # For derivative smoothing
```

```
def plot_titration_curve(df, params, output_dir='output', filename='titration_curve.png'):
```

```
    """
```

```

Plot the raw titration curve (volume vs. potential or pH).
"""
Path(output_dir).mkdir(exist_ok=True)

potential = df['potential'].to_numpy()
volume = df['volume'].to_numpy()
pH = 7 - (potential / 59.16) # Convert mV to pH for plotting

plt.style.use('seaborn-v0_8')
fig, ax = plt.subplots(figsize=(8, 6))
ax.plot(volume, pH, 'o-', color='blue', label='Titration Curve (pH)')
ax.set_xlabel('Volume Added (mL)')
ax.set_ylabel('pH (from potential)')
ax.set_title('Titration Curve')
ax.grid(True)
ax.legend()
plt.tight_layout()
plt.savefig(Path(output_dir) / filename, dpi=300)
plt.close()
print(f"Titration curve saved to {output_dir}/{filename}")

def plot_gran_functions(results, params, output_dir='output', filename='gran_functions.png'):
    """
    Plot Gran functions in vertically stacked panels for raw and opt Zones (separate intervals if extended),
    each with linear interval and fit, plus derivative subplot (no hardcoding).
    """
    Path(output_dir).mkdir(exist_ok=True)

    volume = params['volume']
    g1_raw = results['g1'] # Raw g1 from gran_functions
    g1_opt = results['g1_opt'] # Opt g1 from analyzer
    raw_start, raw_end = results['raw_zone']['start'], results['raw_zone']['end']
    opt_start, opt_end = results['opt_zone']['start'], results['opt_zone']['end']

    # Smooth for derivative (on raw g1)
    g1_smooth = savgol_filter(g1_raw, window_length=min(7, len(g1_raw)), polyorder=2)

```

```

dg1 = np.gradient(g1_smooth, volume)

# Raw/opt metrics
raw_r2 = results['raw_zone']['r2']
raw_veq = results['raw_zone']['veq']
raw_fit = results['raw_zone']['fit']
opt_r2 = results['opt_zone']['r2']
opt_veq = results['opt_zone']['veq']
opt_fit = results['opt_zone']['fit']

plt.style.use('seaborn-v0_8')
fig, axs = plt.subplots(3, 1, figsize=(10, 12), sharex=True)

# Top panel: Raw g1 + raw Zone
axs[0].plot(volume, g1_raw, 'o-', color='blue')
axs[0].axvspan(volume[raw_start], volume[raw_end-1], alpha=0.3, color='red', label='Raw Zone')
x_fit_full = np.linspace(0, volume[-1], 100)
y_raw_fit = raw_fit[0] * x_fit_full + raw_fit[1]
axs[0].plot(x_fit_full, y_raw_fit, '-', color='black')
axs[0].set_ylabel('Raw g1 (k5=0)')
axs[0].set_title(f'Raw Zone ( $R^2$ ={{raw_r2:.3f}},  $V_{eq}$ ={{raw_veq:.3f}} mL, {{raw_end-raw_start}} pts)')
axs[0].grid(True)
axs[0].legend()

# Middle panel: Opt g1 + opt Zone
axs[1].plot(volume, g1_opt, 'o-', color='green')
axs[1].axvspan(volume[opt_start], volume[opt_end-1], alpha=0.3, color='orange', label='Opt Zone')
y_opt_fit = opt_fit[0] * x_fit_full + opt_fit[1]
axs[1].plot(x_fit_full, y_opt_fit, '--', color='orange')
axs[1].set_ylabel(f'Opt g1 (k5={{results["opt_zone"]["k5"]:.3f}})')
axs[1].set_title(f'Opt Zone ( $R^2$ ={{opt_r2:.3f}},  $V_{eq}$ ={{opt_veq:.3f}} mL, {{opt_end-opt_start}} pts)')
axs[1].grid(True)
axs[1].legend()

# Bottom panel: Derivative (raw g1, raw Zone for consistency)
axs[2].plot(volume, dg1, 'o-', color='orange', label='dg1/dV (smoothed, raw g1)')

```

```

    axs[2].axhline(y=0, color='gray', linestyle='--', label='Zero Slope')
    axs[2].axvspan(volume[raw_start], volume[raw_end-1], alpha=0.3, color='red')
    axs[2].set_xlabel('Volume Added (mL)')
    axs[2].set_ylabel('dg1/dV')
    axs[2].set_title('Derivative for Raw Zone Debugging (Negative Focus)')
    axs[2].grid(True)
    axs[2].legend()
    min_dg1 = np.min(dg1)
    axs[2].set_ylim([min_dg1, 0])

    plt.tight_layout()
    plt.savefig(Path(output_dir) / filename, dpi=300)
    plt.close()
    print(f"Gran plot (separate raw/opt Zones) saved to {output_dir}/{filename}")

def plot_screened_k(results, params, output_dir='output', filename='k_screening.png'):
    """
    Plot screened k-values for g1 (if available).
    """
    if 'g1_screened' not in results:
        print("No screened k data available. Skipping plot.")
        return

    Path(output_dir).mkdir(exist_ok=True)

    volume = params['volume']
    screened = results['g1_screened']

    plt.style.use('seaborn-v0_8')
    fig, ax = plt.subplots(figsize=(10, 6))
    colors = plt.cm.viridis(np.linspace(0, 1, len(screened)))
    for i, (k_str, g1_k) in enumerate(screened.items()):
        k_val = float(k_str.split('=')[1])
        ax.plot(volume, g1_k, '-', color=colors[i], label=f'k={k_val}')
    ax.set_xlabel('Volume Added (mL)')
    ax.set_ylabel('g1 (screened k)')

```

```

    ax.set_title('Gran Plot for Screened k Values')
    ax.grid(True)
    ax.legend(bbox_to_anchor=(1.05, 1), loc='upper left')
    plt.tight_layout()
    plt.savefig(Path(output_dir) / filename, dpi=300)
    plt.close()
    print(f"k screening plot saved to {output_dir}/{filename}")

def visualize_all(df, params, results, output_dir='output'):
    """
    Orchestrate all plots: titration curve, Gran functions (with comparison), screening.
    """
    plot_titration_curve(df, params, output_dir)
    plot_gran_functions(results, params, output_dir)
    plot_screened_k(results, params, output_dir)
    print(f"All visualizations saved to {output_dir}")

# Example usage (for testing)
if __name__ == "__main__":
    import pandas as pd
    df = pd.read_csv('data.dat', names=['volume', 'potential'], sep='\s+')
    params = {'V': 25.0}
    # Mock results (replace with analyzer call)
    mock_results = {
        'g1': np.random.rand(len(df)),
        'g1_opt': np.random.rand(len(df)) * 10,
        'raw_zone': {'r2': 0.99, 'fit': (0.5, 2.0), 'veq': 4.0, 'start': 5, 'end': 15, 'num_points': 10},
        'opt_zone': {'r2': 0.995, 'fit': (0.48, 1.92), 'veq': 4.0, 'start': 3, 'end': 18, 'num_points': 15, 'k5': 2.5}
    }
    visualize_all(df, params, mock_results)

```

reporter.py

```
print(open('reporter.py').read())
```

```
# reporter.py: Module 6 for GrANTED - Results Export and Reporting
```

```
import pandas as pd
from pathlib import Path
import json
from reportlab.lib.pagesizes import letter
from reportlab.platypus import SimpleDocTemplate, Paragraph, Spacer, Table
from reportlab.lib.styles import getSampleStyleSheet, ParagraphStyle
from reportlab.lib.units import inch
import numpy as np
import pandas as pd

def convert_to_serializable(obj):
    """
    Recursively convert numpy arrays, scalars, tuples, and pandas Series to lists/primitive types for JSON serialization.
    """
    if isinstance(obj, dict):
        return {k: convert_to_serializable(v) for k, v in obj.items()}
    elif isinstance(obj, (list, tuple)):
        return [convert_to_serializable(item) for item in obj]
    elif isinstance(obj, (np.ndarray, pd.Series)):
        return [convert_to_serializable(item) for item in obj]
    elif isinstance(obj, (np.integer, np.int64, np.int32, np.int16)):
        return int(obj)
    elif isinstance(obj, (np.floating, np.float64, np.float32, np.float16)):
        return float(obj)
    elif isinstance(obj, np.bool_):
        return bool(obj)
    else:
        return obj

def export_to_csv(results, params, output_dir='output', filename='gran_results.csv'):
    """
```

```

Export analysis results to CSV, including raw and opt Zone comparisons.
"""
Path(output_dir).mkdir(exist_ok=True)

# Prepare export data with comparison (raw = unopt baseline, opt = extended)
export_data = {
    'Parameter': [
        'V (mL)', 'C_B (M)', 'titration_type',
        'Raw Zone k5', 'Raw Zone R2', 'Raw Zone V_eq (mL)', 'Raw Zone Num Points',
        'Opt Zone k5', 'Opt Zone R2', 'Opt Zone V_eq (mL)', 'Opt Zone Num Points',
        'Raw Interval Start', 'Raw Interval End', 'Opt Interval Start', 'Opt Interval End'
    ],
    'Value': [
        params.get('V', 25.0),
        params.get('C_B', 0.1),
        params.get('titration_type', 'weak_acid'),
        0.0, # Fixed for raw
        results['raw_zone']['r2'],
        results['raw_zone']['veq'],
        results['raw_zone']['num_points'],
        results['opt_zone']['k5'],
        results['opt_zone']['r2'],
        results['opt_zone']['veq'],
        results['opt_zone']['num_points'],
        results['raw_zone']['start'],
        results['raw_zone']['end'],
        results['opt_zone']['start'],
        results['opt_zone']['end']
    ]
}
df_export = pd.DataFrame(export_data)

filepath = Path(output_dir) / filename
df_export.to_csv(filepath, index=False)
print(f"CSV report saved to {filepath}")

```

```

def export_to_pdf(results, params, output_dir='output', filename='gran_report.pdf'):
    """
    Export summary to PDF report, including raw/opt comparisons.
    """
    Path(output_dir).mkdir(exist_ok=True)

    filepath = Path(output_dir) / filename
    doc = SimpleDocTemplate(str(filepath), pagesize=letter)
    styles = getSampleStyleSheet()
    styles.add(ParagraphStyle(name='CustomTitle', fontSize=14, alignment=1))

    story = []
    story.append(Paragraph("GranTED Titration Analysis Report", styles['Title']))
    story.append(Spacer(1, 12))

    # Parameters table with raw/opt
    param_data = [
        ['Parameter', 'Value'],
        ['V (mL)', str(params.get('V', 25.0))],
        ['C_B (M)', str(params.get('C_B', 0.1))],
        ['titration_type', str(params.get('titration_type', 'weak_acid'))],
        ['Raw Zone R2', f"{results['raw_zone']['r2']:.3f}"],
        ['Raw Zone Veq', f"{results['raw_zone']['veq']:.3f} mL"],
        ['Opt Zone k5', f"{results['opt_zone']['k5']:.3f}"],
        ['Opt Zone R2', f"{results['opt_zone']['r2']:.3f}"],
        ['Opt Zone Veq', f"{results['opt_zone']['veq']:.3f} mL"],
        ['Raw Interval', f"{results['raw_zone']['start']}-{results['raw_zone']['end']}"],
        ['Opt Interval', f"{results['opt_zone']['start']}-{results['opt_zone']['end']}"]
    ]
    story.append(Table(param_data, colWidths=[2*inch, 1.5*inch]))
    story.append(Spacer(1, 12))

    # Green metrics (placeholder)
    story.append(Paragraph("Green Metrics (Placeholder)", styles['Heading2']))
    story.append(Paragraph("Waste: Low | Energy: Minimal | Toxicity: Safe", styles['Normal']))

```

```

doc.build(story)
print(f"PDF report saved to {filepath}")

def save_method_json(params, results, output_dir='output', filename='method.json'):
    """
    Save raw/opt method as JSON for reuse.
    """
    Path(output_dir).mkdir(exist_ok=True)

    # Convert non-serializable types
    method_data = convert_to_serializable({
        'params': params,
        'raw_zone': results['raw_zone'],
        'opt_zone': results['opt_zone'],
        'g1': results['g1'],
        'g1_opt': results['g1_opt']
    })

    filepath = Path(output_dir) / filename
    with open(filepath, 'w') as f:
        json.dump(method_data, f, indent=4)
    print(f"Method JSON saved to {filepath}")

def generate_report(df, params, results, output_dir='output'):
    """
    Orchestrate all exports: CSV, PDF, JSON.
    Args:
        df (pd.DataFrame): Raw data.
        params (dict): From preprocess.py.
        results (dict): From analyzer.py.
        output_dir (str): Directory to save reports.
    """
    export_to_csv(results, params, output_dir)
    export_to_pdf(results, params, output_dir)
    save_method_json(params, results, output_dir)
    print(f"Full report generated in {output_dir}")

```

```

# Example usage (for testing)
if __name__ == "__main__":
    import pandas as pd
    df = pd.read_csv('data.dat', names=['volume', 'potential'])
    params = {'V': 25.0}
    # Mock results (replace with analyzer call)
    mock_results = {
        'optimized': {'k5': 0.98, 'r2': 0.995, 'fit': (0.5, 2.0), 'veq': 4.0, 'num_points': 15},
        'unoptimized': {'r2': 0.99, 'fit': (0.48, 1.92), 'veq': 4.0, 'num_points': 15},
        'interval': (5, 15),
        'g1': np.random.rand(20)
    }
    generate_report(df, params, mock_results)

```